Using Google Routes API in Excel

A brief guide for beginners

Instructions provided by Joshua Alstatt

What is an API?

Simply put, an Application Programming Interface, or API, is a set of instructions exchanged between applications. These instructions are encoded in JSON and delivered via HTTP protocol over the internet. In many ways, APIs are the building blocks that make the modern internet possible. Have you ever made a payment on a smartphone? Held a meeting on Slack? An API was in the background the entire time.

API Keys

API Keys are essentially what they sound like. It is a unique identifier to authenticate software before accessing potentially sensitive data. Treat API keys like the password to your computer. Do not share them, and do not keep the full key in the code itself.

Google Routes

Many tech companies create APIs designed for public use. This is the case for the API that powers Google Maps, called Google Routes. Tech companies are happy to provide such tools, as offering them at a low price point discourages users from scraping this data directly from Google Maps. The first 1000 uses of the Routes API are free. After this, you are charged $15 for another 1000 uses.

Developers may use the Routes API natively in a variety of different programming languages, such as Go, Python, and Java. However, to get Routes to function in Excel, we will need to do a little bit of work in VBA, Microsoft's go-to programming language. If you are not familiar with coding in VBA, I will walk you through this example. The full code will be provided at the bottom.

Accessing the Routes API

1- Open Google Cloud Console (https://console.cloud.google.com/welcome)
   This is where Google stores its APIs for public use.
2- Creating a new project
   a. Click “Open Project Picker” located in the top bar, or press CTRL+O
   b. Click “New Project” and rename
3- In the “Quick Access” Menu, click “APIs and Services”
4- Search “Routes API” from Google Enterprise API
5- Click “Manage API”
6- Create a billing account (free for first 1000 count. $15/1000 count afterwards)
7- Set API restrictions
   API restrictions put strict limits on how the API can be accessed. You can also restrict usage to specific users or a maximum number of requests. For this exercise, this step can be skipped, but Restrictions are highly recommended in most contexts.
8- Enable API
9- Get API Key
   a. Click “Keys and Credentials” in the menu on the left-hand side of the screen.
   b. In the “API Keys” table, find the “Maps Platform API Key” under the Name column
   c. Click “Show Key”, located at the last column on the same row.
   d. Copy the key

Setting Up Excel

If the “Developer” tab is missing, go to File, then Options in the bottom left. Click “Customize Ribbon” and then check “Developer” in the list of options on the right.

10- Open Microsoft Excel
11- Create Module
   a. Click the Developer Tab
   b. Click “Visual Basic” on the left. This will open a new workbook in Microsoft Visual Basic for Applications
   c. In the new book, click “Insert”
   d. Click “Module”
12- While still in the VBA Editor
   a. Click “Tools”
   b. Click “References”
   c. Check “Check Microsoft Scripting Runtime”
   d. Check “Check Microsoft XML V.6”. This allows Visual Basic to sort information into JSON and XML formats, which VBA would otherwise not be able to do.

Coding in VBA

Setup & Option Explicit

We are now ready to start coding. Let's walk through the basic parts, starting with our setup. Option Explicit means all variables must be defined explicitly and not assumed. This can speed up the code and reduce the amount of memory required.

Public Function is defining "GetRouteDistance" as a function that Excel can recognize. Its format is GetRouteDistance(Origin,Destination,Key). So if cells A1, B1, C1 contain the Origin, Destination, and API Key respectively, then you would input "=GetRouteDistance(A1, B1, C1)" into any other cell to return the driving distance between the two points.

The rest of this section defines the terms that we will need later. This is necessary because "Option Explicit" is called out.

```
Option Explicit

Public Function GetRouteDistance(Origin As String, Destination As String, API_Key As String) As
Variant
    Dim xmlhttp As Object
    Dim url As String
    Dim payload As String
    Dim response As String
    Dim posText As Long, posValue As Long
```

Endpoints

Earlier I mentioned that an API is a set of instructions. The Endpoint is where those instructions are delivered. The HTTP Object is the route by which the instructions are sent.

```
    url = https://routes.googleapis.com/directions/v2:computeRoutes
    Set xmlhttp = CreateObject("MSXML2.ServerXMLHTTP.6.0")
```

JSON Payload

A JSON Payload is the "instructions" that are being sent. JavaScript Object Notation, or JSON, is a standard text-based format used by computers to store and transmit data. "On Error" tells the code what to do if there is a problem. ErrorHandler code will be at the very bottom of the function.

```
    payload = "{" & _
                """origin"":{""address"":""" & Origin & """}," & _
                """destination"":{""address"":""" & Destination & """}," & _
                """travelMode"":""DRIVE""," & _
                """routingPreference"":""TRAFFIC_AWARE""," & _
                """units"":""IMPERIAL""" & _
              "}"
    On Error GoTo ErrorHandler
```

POST Request

By this point, Excel understands what we are asking, and Visual Basic has formatted those instructions in a way that Google Maps can understand. By converting JSON text into a format that is easily read by a machine, we also avoid creating dependency on Google's external library. The POST request sends the packet to the server for Google to read.

In VBA, it is good practice to set our variables to "Nothing" when we are done with them. This frees up memory and prevents circular references.

```
    With xmlhttp
        .Open "POST", url, False
        .setRequestHeader "Content-Type", "application/json"
        .setRequestHeader "X-Goog-Api-Key", API_Key
        ' FieldMask dictates what data Google sends back (saves bandwidth and costs)
        .setRequestHeader "X-Goog-FieldMask", "routes.distanceMeters,routes.duration"
        .send payload
        response = .responseText
    End With
     posText = InStr(response, """distanceMeters"":")
     If posText > 0 Then
        posValue = posText + Len("""distanceMeters"":")
        ' Extract numbers following the key and convert meters to miles
        GetRouteDistance = Round(Val(Mid(response, posValue)) * 0.000621371, 2)
    Else
        GetRouteDistance = "Route Not Found"
    End If

    Set xmlhttp = Nothing
```

ErrorHandler

ErrorHandler lies at the very bottom of the code. If there is a problem and the function wants to crash, ErrorHandler first asks the code *why* it's crashing, and then reports the error to the user. It is a simple yet effective way to troubleshoot when your code does not run as expected.

```
ErrorHandler:

GetRouteDistance = "Error: " & Err.Description

Set xmlhttp = Nothing
```

Be sure to save the Excel sheet as "macro enabled" (.xlsm) before exiting the VBA editor.

Sheet Setup

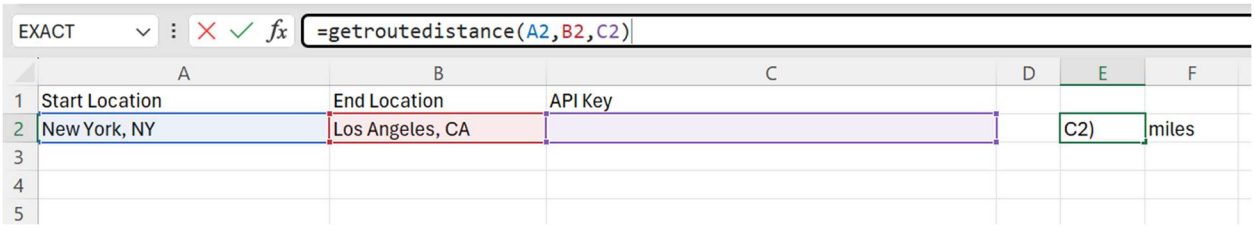
With the hard part out of the way, the only thing left to do is enter our variables and test the code. Below is how I have organized my sheet, but it can be in any format if the formula is consistent.

Cell A2 contains the start location.

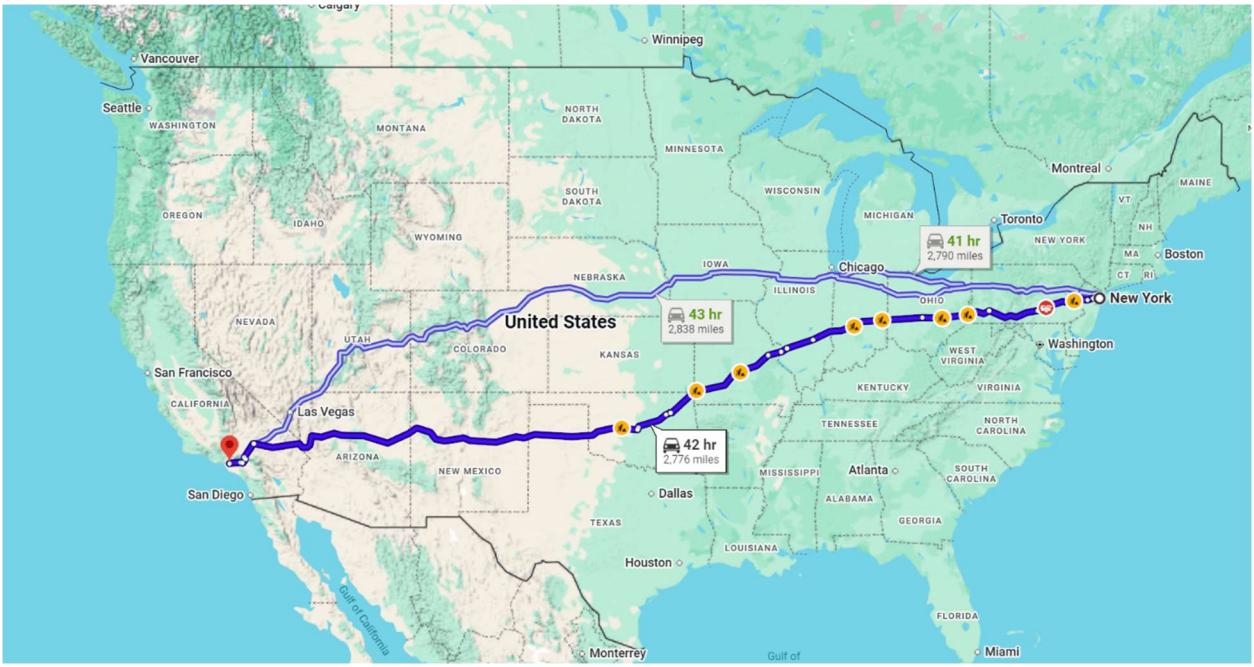
Cell B2 contains the destination.

Cell C2 contains your API Key (Mine is hidden for this example)

Cell E2 contains the function “=getroutedistance(A2,B2,C2)”. The result will display a number in miles.



Finally, check the output against Google Maps to verify accuracy. Google Routes API will always choose the shortest driving distance between two points, regardless of which route is recommended by Google Maps.



VBA Code

```
Option Explicit

' Adds Excel function to get driving distance using Google Routes API
Public Function GetRouteDistance(Origin As String, Destination As String, API_Key As String) As
Variant
    Dim xmlhttp As Object
    Dim url As String
    Dim payload As String
    Dim response As String
    Dim posText As Long, posValue As Long

    ' Google Routes API v2 Endpoint
    url = https://routes.googleapis.com/directions/v2:computeRoutes

    ' Set up HTTP Object
    Set xmlhttp = CreateObject("MSXML2.ServerXMLHTTP.6.0")

    ' Construct JSON Payload
    payload = "{" & _
                """origin"":{""address"":""" & Origin & """}," & _
                """destination"":{""address"":""" & Destination & """}," & _
                """travelMode"":""DRIVE""," & _
                """routingPreference"":""TRAFFIC_AWARE""," & _
                """units"":""IMPERIAL""" & _
              "}"
    On Error GoTo ErrorHandler

    ' Configure and send POST request
    With xmlhttp
        .Open "POST", url, False
        .setRequestHeader "Content-Type", "application/json"
        .setRequestHeader "X-Goog-Api-Key", API_Key
        ' FieldMask dictates what data Google sends back (saves bandwidth and costs)
        .setRequestHeader "X-Goog-FieldMask", "routes.distanceMeters,routes.duration"
        .send payload
        response = .responseText
    End With
' Lightweight JSON parsing via string manipulation (avoids external library dependency)
     posText = InStr(response, """distanceMeters"":")
     If posText > 0 Then
        posValue = posText + Len("""distanceMeters"":")
        ' Extract numbers following the key and convert meters to miles
        GetRouteDistance = Round(Val(Mid(response, posValue)) * 0.000621371, 2)
    Else
        GetRouteDistance = "Route Not Found"
    End If

    Set xmlhttp = Nothing

    Exit Function

ErrorHandler:

GetRouteDistance = "Error: " & Err.Description

Set xmlhttp = Nothing

End Function
```